Models in ASP.NET Core Web API

Back to ASP.NET Core Web API Tutorials

Models in ASP.NET Core Web API

In ASP.NET Core Web API, models are used to represent the business logic and manage data within an application. They encapsulate the data structures and business rules that govern how data is processed and stored. Models in ASP.NET Core Web API typically define entities, relationships, and validation rules to ensure data integrity and consistency across the application.

What are Models in ASP.NET Core Web API?

In ASP.NET Core Web API, models are used to represent the data and business logic of the application. They define the structure of the data that is exchanged between the client and the server, and they can also include business rules and validation rules to ensure that the data is in the correct format and meets the business requirements.

Key Features of Models in ASP.NET Core Web API

The following are the key features of Models in ASP.NET Core Web API:

Data Representation (POCOs): Models use Plain Old CLR Objects, which ensure that they are independent of any data store or framework dependencies, making them lightweight and easy to manage.

Relationships: Models can define relationships between different entities, such as one-to-many or many-to-many relationships, using navigation properties.

Validation: Models can include validation rules to ensure that the data is in the correct format and meets the business requirements. This can be done using data annotations or custom validation logic.

Inheritance: Models can inherit from other models, allowing for the reuse of common properties and behaviors.

Serialization: Models can be serialized and deserialized to and from JSON or other formats, enabling data exchange between the client and the server.

Data Validation with Data Annotations

Models can be enhanced with attributes from the System.ComponentModel.DataAnnotations namespace to apply validation rules directly to the properties. Common validation attributes include:

Common Data Annotations:

- [Required]: Ensures a property is not null or empty.
- [StringLength(max)]: Limits the length of a string.
- [Range(min, max)]: Specifies a numeric range.
- [RegularExpression(pattern)]: Validates against a regular expression.

This approach helps enforce the integrity of incoming data.

```
public class Product
{
    [Required]
    [StringLength(100, ErrorMessage = "Name cannot exceed 100 characters")]
    public string Name { get; set; }

    [Range(0.01, 10000.00, ErrorMessage = "Price must be between 0.01 and 10000.00")]
    public decimal Price { get; set; }

    [Required]
    [EmailAddress]
    public string ContactEmail { get; set; }
}
```

Data Validation with Fluent Validation

Fluent Validation offers a more flexible and maintainable approach to validation compared to data annotations. It allows you to define validation rules in a separate class, keeping the validation logic organized and easy to manage. Here's how you can use Fluent Validation with models:

```
public class ProductValidator : AbstractValidator<Product>
{
    public ProductValidator()
    {
        RuleFor(p => p.Name).NotEmpty().Length(1, 100);
        RuleFor(p => p.Price).GreaterThan(0.01).LessThan(10000.00);
        RuleFor(p => p.ContactEmail).NotEmpty().EmailAddress();
    }
}
```

Relationships in Models

Models can define relationships between different entities using navigation properties. These properties allow you to navigate from one entity to another, facilitating the management of related data.

```
public class Order
{
    public int OrderId { get; set; }
    public int CustomerId { get; set; }
    public Customer Customer { get; set; }
    public ICollection<OrderItem> OrderItems { get; set; }
}

public class OrderItem
{
    public int OrderItemId { get; set; }
    public int OrderId { get; set; }
    public Order Order { get; set; }
    public int ProductId { get; set; }
    public Product Product { get; set; }
}
```

Explanation:

- Navigation Properties: Properties like Customer and OrderItems allow navigation between related entities.
- Collections: Collections like OrderItems can represent one-to-many relationships.

Inheritance in Models

Models can inherit from other models, allowing for the reuse of common properties and behaviors. This can be useful when dealing with entities that share common attributes.

```
public class BaseEntity
{
    public int Id { get; set; }
    public DateTime CreatedDate { get; set; }
    public DateTime? ModifiedDate { get; set; }
}

public class Product : BaseEntity
{
    public string Name { get; set; }
    public decimal Price { get; set; }
}

public class Customer : BaseEntity
{
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public string Email { get; set; }
}
```

Explanation:

- Base Class: BaseEntity provides common properties for all derived classes.
- Derived Classes: Product and Customer inherit from BaseEntity, ensuring consistency in the data structure.

Serialization in Models

Models can be serialized and deserialized to and from JSON or other formats, enabling data exchange between the client and the server. This is important for transmitting data between different components of the application.

```
public class Product
{
    public int ProductId { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
    public bool IsAvailable { get; set; }
    public DateTime? ExpiryDate { get; set; }
}
```

Explanation:

- Serialization: The Product class can be serialized into JSON for transmission between the server and client.
- Deserialization: The JSON data can be deserialized back into the Product class when received by the server.

Conclusion

Models in ASP.NET Core Web API are essential for managing data and business logic in a structured and maintainable way. They provide a clear framework for data representation, relationships, validation, inheritance, and serialization, ensuring that the application can effectively handle data while maintaining high standards of quality and performance.

ASP.NET Core Web API Tutorial
ASP.NET Core Web API Tutorial
ASP.NET Core Web API Tutorial
Introduction to ASP.NET Core Web API
Prerequisites to Learn ASP.NET Core Web API
Advantages and Disadvantages of ASP.NET Core Web API
Disadvantages of ASP.NET Core Web API
Controller in ASP.NET Core Web API
Controller in ASP.NET Core Web API
Controller Action Methods in ASP.NET Core Web API
Controller Action Methods in ASP.NET Core Web API
Controller Action Methods in ASP.NET Core Web API
Routing in ASP.NET Core Web API
Routing in ASP.NET Core Web API
Routing in ASP.NET Core Web API
Models in ASP.NET Core Web API
Models in ASP.NET Core Web API
Models